

小乔智投-架构与联调流程汇报

1. 汇报结论

小乔智投当前架构不是单一 chatbot，而是“前端统一 Chat 入口 + 小乔后端任务运行时 + Windows 测试机 Agent 执行器”的三段式系统。

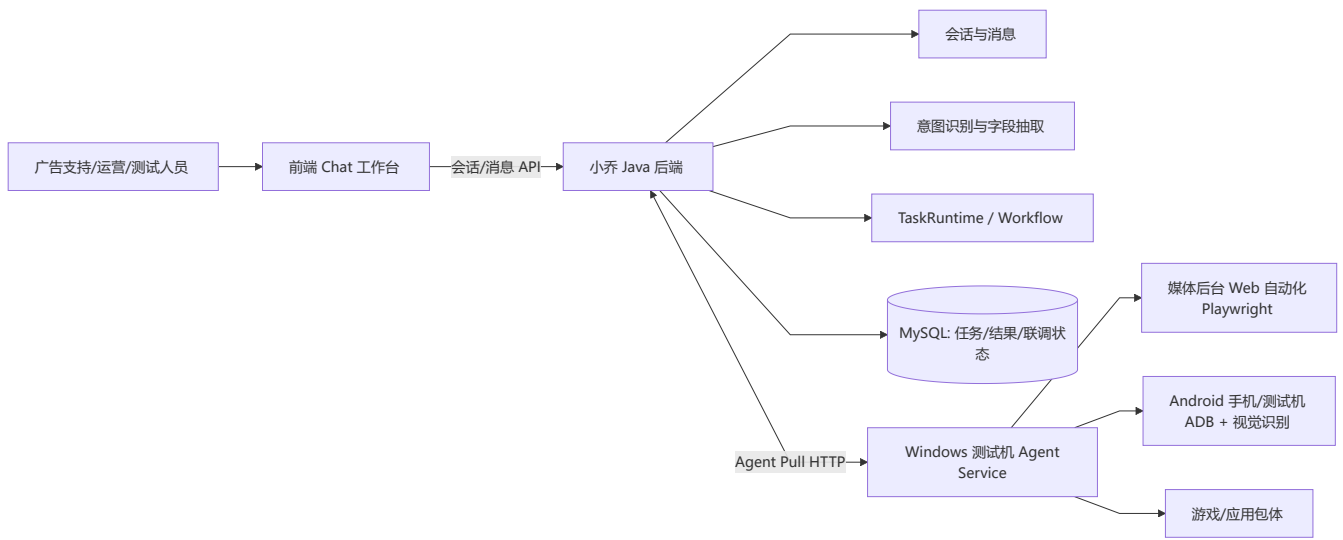
前端负责承接人的自然语言输入和任务过程展示；小乔 Java 后端负责会话、意图识别、字段抽取、workflow 编排、任务状态和结果沉淀；Windows 测试机上的 Python Agent 负责真实操作媒体后台、手机渠道和游戏包体，把自动联调过程以进度、截图、日志和最终结果回传给后端。

这套设计的关键价值是把“人手工做联调”的过程拆成可编排、可观测、可替换的模块：媒体平台、渠道入口、游戏/应用处理都以插件方式接入。当前已落地的样例组合是：

- 媒体平台：巨量引擎 `oceanengine`
- 渠道入口：抖音 `douyin`
- 游戏处理：三国杀 `sanguosha`

后续扩展到其他媒体、渠道或游戏时，不需要重写整条联调链路，只需要新增对应插件并注册到执行 pipeline。

2. 总体架构



模块关系

模块	主要职责	当前实现位置
前端 Chat 工作台	用户提问、展示小乔回复、展示任务状态、联调步骤、截图和结果	当前以 API 契约承接，后端已提供会话和任务接口
小乔 Java 后端	会话管理、意图路由、workflow 编排、任务/结果存储、自动联调任务调度	<code>xiaoqiao-mvp-java</code>
Windows 测试机 Agent	常驻轮询任务、执行联调、控制浏览器和手机、上报进度与结果	<code>xiaoqiao-agent-service</code>

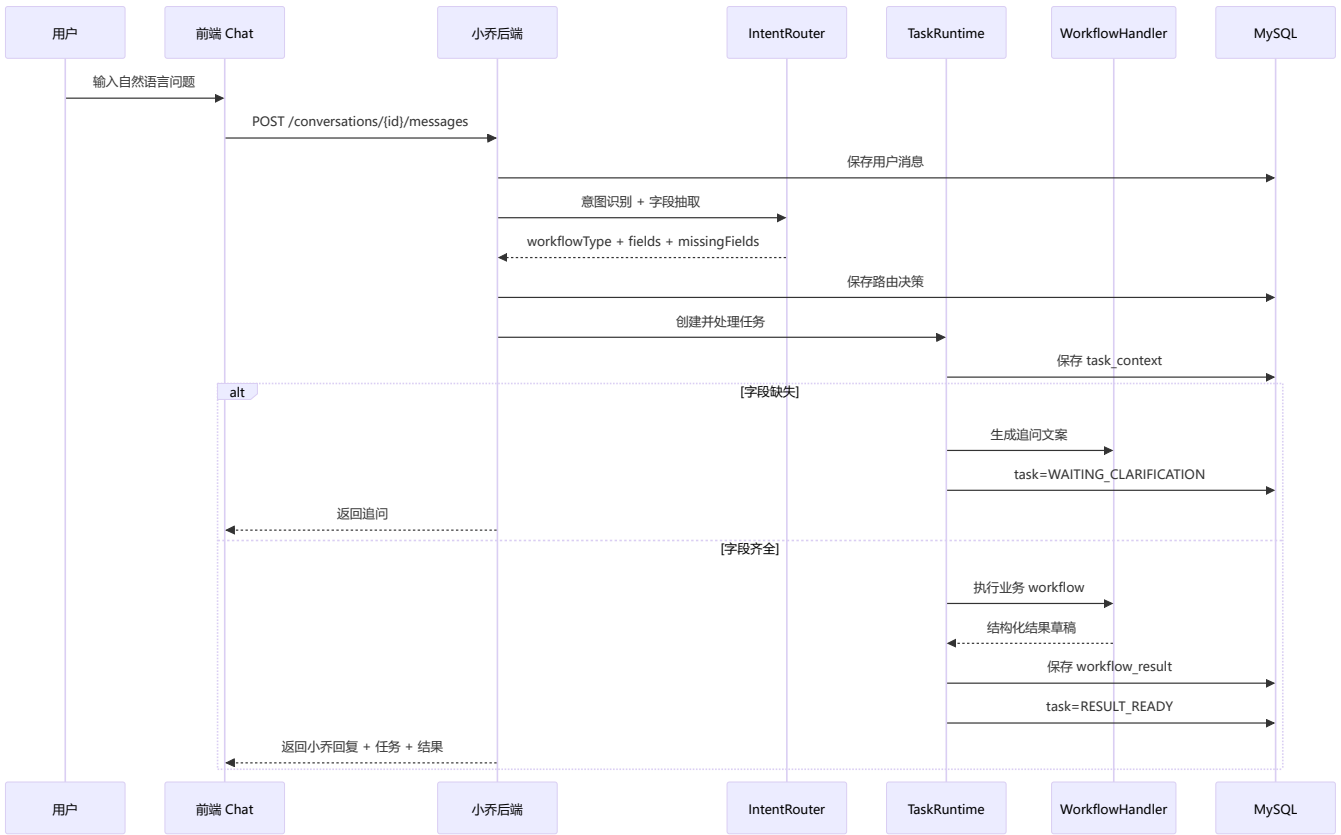
模块	主要职责	当前实现位置
媒体 SOP 插件	登录媒体后台、进入联调工具、生成二维码或测试链接、轮询转化事件	plugins/media_sops
渠道 SOP 插件	手机端扫码、授权、刷 feed、查找广告、点击 CTA	plugins/channel_sops
游戏 Handler 插件	启动游戏、登录账号、完成指定游戏内动作	plugins/game_handlers

3. 前端 Chat 到后端 workflow 的主流程

前端只面对“小乔”一个入口，后台根据用户输入自动识别具体业务流。当前 Java MVP 支持四类 workflow：

- `HELP`：使用帮助
- `DEMAND`：需求沟通
- `DIAGNOSIS`：问题排查
- `DEBUGGING`：广告联调

主链路如下：



后端核心链路为：

```
API -> ConversationService -> IntentRouter -> TaskRuntime -> WorkflowHandler -> WorkflowResult
```

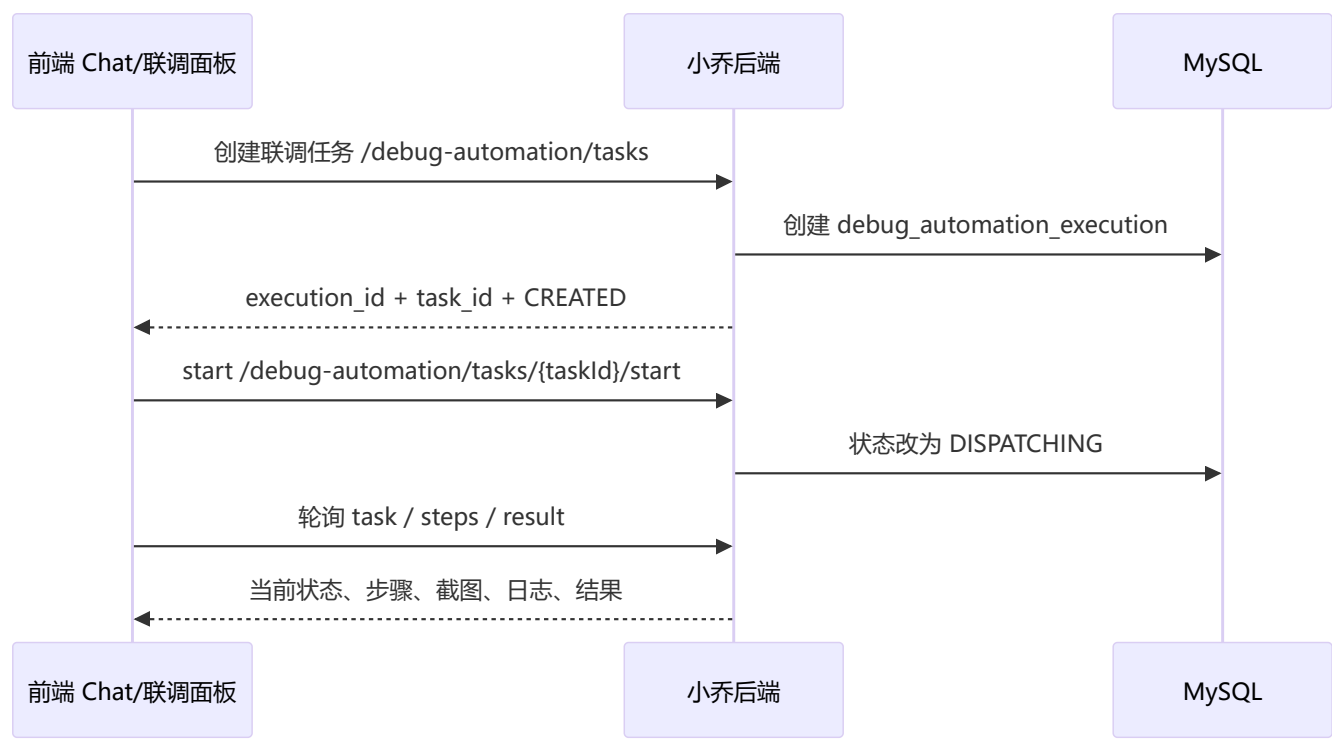
其中 `IntentRouter` 采用“LLM 优先、规则兜底”的策略：如果 Qwen / LiteLLM 可用，则用模型做意图识别和字段抽取；如果模型不可用、超时或输出不合法，则自动回退到规则路由。LLM 只参与路由和摘要润色，不直接控制任务状态，也不直接调用 Windows 测试机。

4. 自动联调 workflow 流程

广告联调是当前架构中最能体现“Chat + 后端编排 + 本地执行器”的流程。它分成两条链：

- 用户侧链路：从 Chat 触发联调任务，并在前端持续查看状态。
- Agent 侧链路：Windows 测试机主动向后端拉取任务，执行并回传过程。

4.1 用户侧调度流程

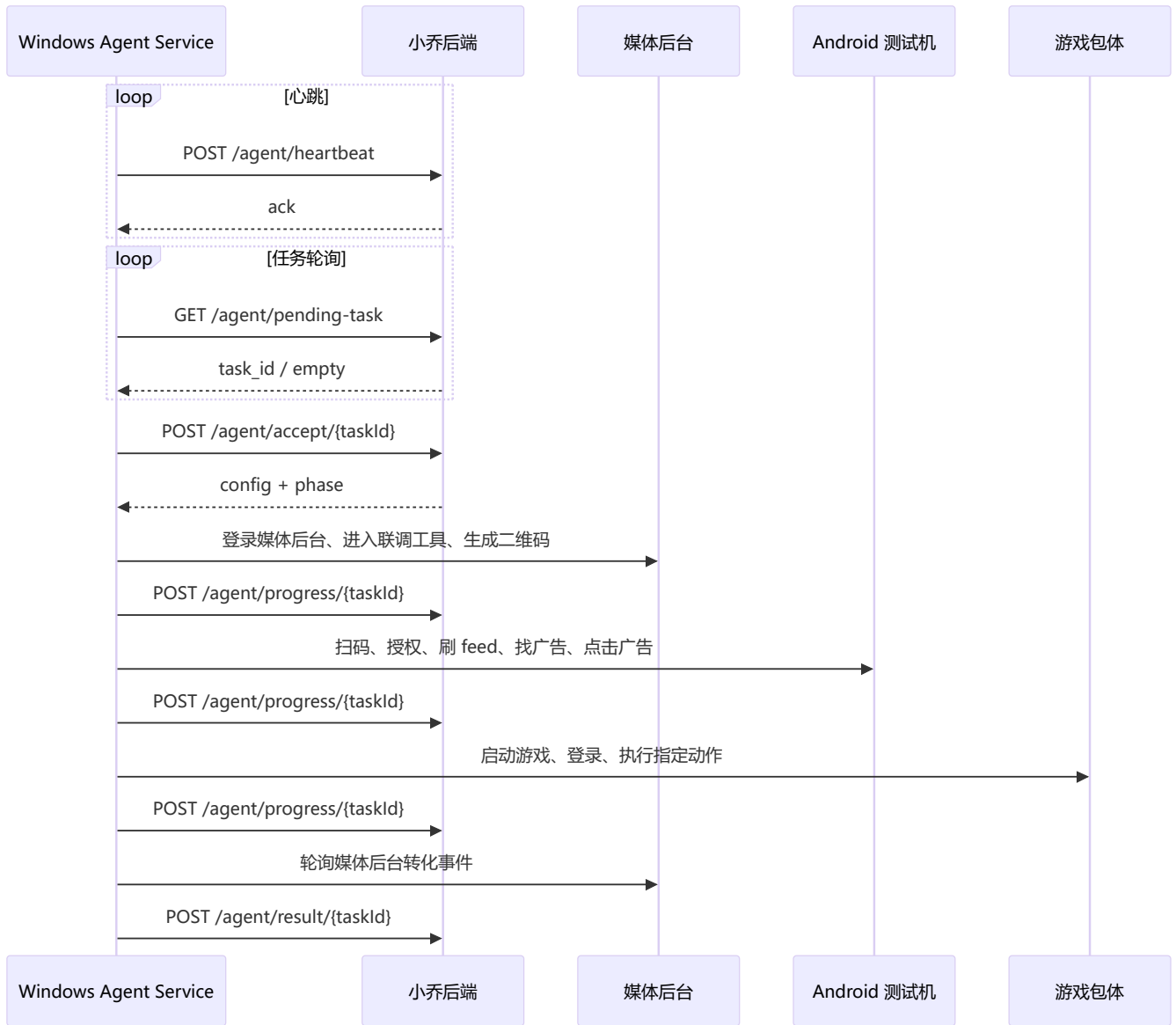


后端保存的关键对象包括：

- `debug_automation_executions`：一次联调执行的总状态、当前阶段、当前步骤、Agent、控制信号、错误信息。
- `debug_execution_steps`：每个执行步骤的 phase、step、状态、截图、日志和错误信息。
- `tasks / workflow_results`：Chat workflow 的任务和结构化结果。

4.2 Windows Agent 执行流程

Windows Agent 采用 pull 模式，不要求后端主动连入测试机。这适合办公网、测试机网络策略复杂、手机和浏览器只在本地可控的场景。



- Agent 常驻后主要做三件事：
- 1. 心跳：上报 `agent_id`、`busy/idle`、当前任务、ADB 设备状态。
 - 2. 拉任务：没有任务执行时轮询后端 pending task，拿到任务后 accept。
 - 3. 执行 pipeline：按媒体、渠道、游戏插件组合执行，并把进度、截图、日志、错误、最终结果回传。

4.3 联调执行步骤

当前 pipeline 将自动联调拆成三段：

阶段	步骤	说明
Web	login	Playwright 打开媒体后台并登录
Web	navigate_event	进入事件资产或联调工具页面
Web	handle_popup	处理弹窗、异常提示
Web	enter_debug_tool	进入联调向导

阶段	步骤	说明
Web	select_channel	选择目标渠道号
Web	generate_qr	生成二维码或测试链接，并准备给手机使用
Mobile	scan_qr	使用 ADB 打开渠道 App 并扫码
Mobile	authorize	完成授权确认并返回 feed
Mobile	find_ad	刷新 feed、滑动查找目标广告
Mobile	click_ad	点击广告 CTA，触发下载/打开
Mobile	launch_game	等待或主动启动游戏
Mobile	login_game	游戏内账号密码登录
Polling	poll_events	回到媒体后台轮询转化事件是否成功

状态机覆盖的主要状态包括：

```
CREATED -> WAITING_CONFIRM -> DISPATCHING
-> RUNNING_WEB_PREPARE
-> RUNNING_MOBILE_SCAN
-> RUNNING_MOBILE_FIND_AD
-> RUNNING_MOBILE_LAUNCH
-> RUNNING_SUCCESS_POLL
-> CONSTRUCTING_DATA
-> SUCCESS / FAILED / PAUSED / MANUAL_TAKEOVER / AGENT_UNREACHABLE
```

前端可以基于这些状态展示“当前做到哪一步、是否可暂停、是否可接管、失败在哪里、证据截图是什么”。

5. 控制与可观测性

自动联调不是黑盒脚本，而是可控制、可恢复、可审计的执行过程。

控制能力

操作	API	说明
暂停	<code>POST /debug-automation/tasks/{taskId}/pause</code>	后端写入 <code>pause</code> 控制信号，Agent 在检查点暂停
恢复	<code>POST /debug-automation/tasks/{taskId}/resume</code>	后端写入 <code>resume</code> 控制信号，Agent 继续执行
人工接管	<code>POST /debug-automation/tasks/{taskId}/takeover</code>	后端写入 <code>cancel</code> ，执行状态进入人工接管

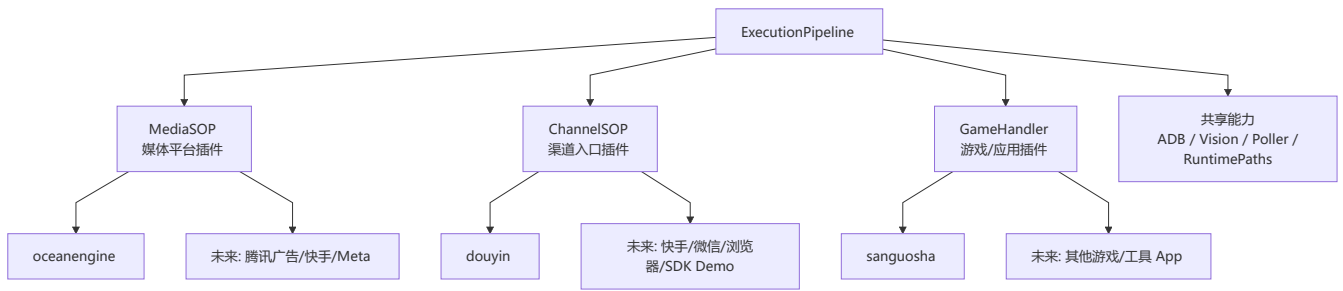
观测能力

类型	说明
任务总状态	当前执行阶段、Agent、错误原因、创建/更新时间
步骤明细	每一步的 phase、step_name、step_index、status
截图证据	Agent 上传 base64 截图，后端保存为文件路径
日志摘要	每一步可带 log_lines，便于还原执行过程
最终结果	success / failed / cancelled，以及事件轮询结果

这使得联调结果可以被前端还原成“执行时间线”，也可以沉淀成后续排查和复盘证据。

6. 插件化扩展设计

自动联调的扩展点明确分成三类：



6.1 媒体插件 MediaSOP

媒体插件负责“把媒体后台准备好”，典型职责是：

- 登录媒体后台。
- 进入事件资产或转化联调页面。
- 选择应用、广告账户、渠道号、事件。
- 生成二维码、测试链接或其他可传递给手机端的入口。
- 提供轮询转化结果需要的页面上下文。

当前接口：

```
prepare(config, progress_hook) -> PrepareResult
cleanup()
```

新增媒体时，只需要实现新的 MediaSOP，例如 tencent_ads.py、kuaishou.py，并注册为新的媒体名称。

6.2 渠道插件 ChannelSOP

渠道插件负责“在手机渠道里找到并点击广告”，典型职责是：

- 将二维码或链接传递给手机。
- 打开渠道 App。
- 完成扫码、授权、跳转。
- 刷新 feed 或搜索入口。
- 通过视觉模型和 ADB 查找目标广告。
- 点击 CTA 并处理安装、打开、权限确认等流程。

当前接口：

```
find_and_click_ad(prepare_result, config, progress_hook) -> bool  
cleanup()
```

新增渠道时，可以复用 ADB、视觉识别、截图、滑动和刷新检测能力，只替换渠道自身的 SOP。

6.3 游戏插件 GameHandler

游戏插件负责“广告点击后的应用内动作”，典型职责是：

- 启动或等待游戏启动。
- 登录指定账号。
- 处理弹窗、公告、权限。
- 执行游戏内关键行为，如创角、进入大厅、付费测试、关键行为触发。

当前接口：

```
launch_and_login(config, progress_hook) -> bool  
cleanup()
```

新增游戏时，不影响媒体和渠道插件；只要实现游戏自身的登录和关键行为流程即可。

6.4 插件配置方式

一次任务通过配置决定使用哪组插件：

```
{  
  "media": "oceanengine",  
  "channel": "douyin",  
  "game": "sanguosha",  
  "media_config": {},  
  "channel_config": {},  
  "game_config": {},  
  "mobile_env": {},  
  "ai_model": {}  
}
```

`ExecutionPipeline` 根据 `media/channel/game` 动态实例化插件，并向插件注入共享依赖：

- `ADBTools`：手机控制、安装、点击、输入、截图。
- `MiniMaxVision`：视觉模型识别页面元素和广告。
- `SuccessPoller`：媒体后台事件轮询。
- `progress_hook`：统一进度上报。
- `control_check`：暂停、恢复、取消控制检查。

7. 可扩展性价值

这套插件化设计把稳定部分和变化部分分开：

稳定部分	变化部分
Chat 入口、任务模型、状态机、进度回传、截图证据、控制信号	不同媒体后台页面、不同渠道 App 操作、不同游戏登录和行为
后端 API 和数据库结构	插件内部 SOP
Agent 轮询和执行生命周期	具体 Playwright / ADB / Vision 步骤

因此后续扩展时可以按组合增长：

- 接一个新媒体：新增一个 `MediaSOP`。
- 接一个新渠道：新增一个 `ChannelSOP`。
- 接一个新游戏：新增一个 `GameHandler`。
- 接一条新业务 workflow：后端新增 `workflowHandler`，前端继续复用 Chat 工作台和任务展示。

8. 当前边界与下一步

当前已经具备的工程骨架：

- Java 后端已提供会话、消息、任务、workflow result、自动联调任务和 Agent API。
- Agent Service 已具备心跳、拉任务、执行管理、进度回传、控制信号、插件化 pipeline。
- 插件体系已形成媒体、渠道、游戏三层抽象，并有巨量 + 抖音 + 三国杀样例。
- 数据库表已覆盖会话、消息、路由、任务、结果、联调执行和步骤证据。

建议下一步重点补齐：

1. 前端联调面板：把 `debug_automation_executions` 和 `debug_execution_steps` 渲染成执行时间线、截图证据和控制按钮。
2. 联调任务从 Chat 自动创建：将 `DEBUGGING` workflow 的准备项与 `/debug-automation/tasks` 打通，减少人工触发脚本。
3. 插件注册机制配置化：从当前代码注册升级为配置发现或插件 manifest，降低新增媒体/游戏的改动面。
4. 失败分类标准化：沉淀媒体登录失败、设备不可达、广告未找到、游戏启动失败、事件轮询超时等统一错误码。
5. 执行证据归档：将截图、日志、事件轮询结果和最终结论绑定到 workflow result，便于复盘和审计。

